

# L'ODYSSEE DE L'IA

« Je propose de remplacer la question "Les machines peuvent-elles penser ?" par une autre, qui lui est étroitement liée et qui s'exprime en des termes relativement sans ambiguïté. [...] La nouvelle forme du problème peut être décrite en termes d'un jeu que nous appellerons le "jeu de l'imitation". »

« La question initiale, "Les machines peuvent-elles penser ?", est selon moi trop insignifiante pour mériter une discussion. Néanmoins, je crois qu'à la fin du siècle, l'usage des mots et l'opinion générale des personnes instruites auront tellement changé que l'on pourra parler de machines qui pensent sans s'attendre à être contredit. »

Alan Turing, *Computing Machinery and Intelligence* (1950)

Kurt Gödel (« un système ne peut pas prouver sa propre cohérence ») et Alan Turing étaient des mathématiciens et des logiciens, pas des essayistes ; leurs phrases originales sont souvent techniques, arides, parfois provoquantes, et laissent sciemment une forme d'incomplétude que le lecteur doit combler.

La portée du théorème de Gödel déborde la logique : elle établit la fin de l'illusion d'un savoir totalement fermé sur lui-même et prouve qu'aucune pensée ne peut fonder sa propre vérité de l'intérieur. Turing montre lui qu'il est absurde de chercher si la machine pense tant que l'homme est incapable de définir sa propre pensée. Le drame du langage n'est pas de savoir si la machine pense, mais de reconnaître que dès lors que « ça parle », l'humain lui-même attribue une pensée là où il n'y a peut-être que de la structure.

Mais l'Intelligence Artificielle n'est pas née d'un consensus philosophique : là où Gödel et Turing fermaient des portes en traçant les frontières du théorique, d'autres allaient les enfoncer par la pratique.

Le premier geste fondateur remonte à 1943, avec Warren McCulloch et Walter Pitts : en modélisant le neurone par une simple porte logique, ils posent l'hypothèse que la pensée pourrait se ramener à du calcul. L'idée reste alors spéculative, sans langage formel pour la porter jusqu'au bout. Claude Shannon, en 1948, en fondant la théorie de l'information, réussit un tour de force qui dépasse largement son objet initial : il sépare l'information du sens. Peu importe ce que veut dire un message, seul comptent le coût de sa transmission et sa structure statistique : si le signal peut se penser indépendamment de toute signification, alors le neurone-calcul peut, lui aussi, s'affranchir de l'expérience humaine. C'est cette convergence, encore informelle en 1948, que la Conférence de Dartmouth vient institutionnaliser en 1956.

## INTRODUCTION : LES ORIGINES

L'été 1956 à Dartmouth est considéré comme l'acte de naissance officiel de l'intelligence artificielle en tant que champ de recherche autonome.

### Conférence de Dartmouth (1956)

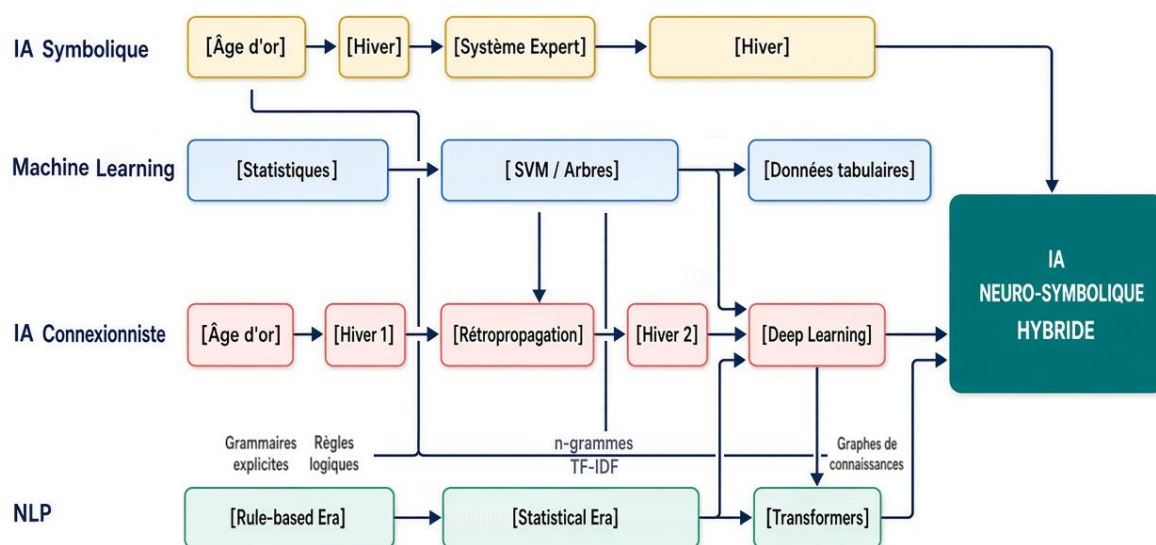
Dartmouth a été un lieu d'effervescence où les deux grands paradigmes de l'IA (symbolique et connexionniste/apprentissage) ont été posés côte à côte.

- **Le raisonnement symbolique** : c'est l'idée de programmer l'intelligence en donnant à la machine des concepts clairs et des règles logiques explicites, comme on le ferait avec un être humain très logique.
  - Influence directe : naissance des systèmes experts, du langage LISP (créé par McCarthy peu après), et des programmes de résolution de problèmes.
  - Réalisation : Systèmes experts des années 1970-1980 (MYCIN pour le diagnostic médical), Langage de programmation LISP et PROLOG.
  - Conséquence : Le raisonnement symbolique a connu un grand succès, puis un échec relatif face à la complexité du monde réel. Il fallait coder manuellement des milliers de règles, et le nombre de combinaisons possibles explosait.
- **Les réseaux de neurones** : En 1943, McCulloch et Pitts ont démontré qu'on pouvait modéliser mathématiquement un neurone capable de réaliser des opérations logiques, prouvant ainsi qu'une machine pouvait reproduire des aspects du raisonnement.
  - Influence : Dartmouth a permis de discuter des réseaux de neurones et de les considérer comme l'un des axes de recherche prioritaires, avec notamment les travaux de Marvin Minsky (modèles neuronaux) et d'Oliver Selfridge (reconnaissance de formes).
  - Réalisation : Ces travaux, dans le sillage de McCulloch et Pitts, annoncent le Perceptron de Frank Rosenblatt (1958), tandis que les idées de Selfridge se prolongeront bientôt dans Pandemonium (1959).
- **L'apprentissage** : Les organisateurs envisagent que la capacité d'apprendre puisse elle-même être programmée : non plus une règle fixée d'avance, mais un comportement qui s'ajuste au contact des données.
  - L'auto-correction par l'expérience : Auto-amélioration (le programme modifie ses propres paramètres internes selon ses succès/ échecs, sans intervention humaine), Apprentissage par renforcement rudimentaire (une forme primitive de récompense / punition guidant le comportement du programme)
  - Réalisation : Ces intuitions, sans lien direct avec les discussions de Dartmouth, trouveront leurs premières formalisations dans le champ statistique : en 1958, David Cox formalise la régression logistique pour la classification binaire (elle devient l'un des piliers de la statistique et de l'apprentissage classique pour estimer des probabilités) ; en 1967, KNN (k-Nearest-Neighbors) est l'un des premiers algorithmes de reconnaissance de formes purement statistiques, sans aucune notion de neurone ni de règle logique.

## Convergence des paradigmes de l'IA

L'intuition connexionniste va rapidement se ramifier. D'un côté, la question de la structure (comment modéliser le neurone et le réseau) donnera naissance à l'IA connexionniste. De l'autre, la question du comportement (comment la machine ajuste ses propres règles au contact de l'expérience) ouvrira la voie à l'IA par apprentissage. À ces deux héritiers d'une même intuition s'ajoute un troisième chemin, radicalement différent : l'IA symbolique.

Ces trois chemins n'ont pas tous le même statut : connexionnisme et apprentissage relèvent d'une même intuition fondatrice, mais leurs trajectoires se sont assez tôt dissociées pour mériter un traitement séparé. Une quatrième histoire se joue en parallèle, à la croisée des trois précédentes : celle du langage.



Encore balbutiante à Dartmouth, le NLP devient alors la branche qui traverse les trois autres plutôt que celle qui les complète à égalité : la traduction automatique a d'abord emprunté au symbolique (règles grammaticales explicites dans les années 1950-1960), puis, des décennies plus tard, à l'apprentissage statistique (traduction automatique statistique dans les années 1990-2000), et au connexionniste (réseaux de neurones et transformers ensuite).

Si les réseaux de neurones profonds ont fait la preuve de leur efficacité remarquable pour l'extraction de motifs complexes et la reconnaissance de formes à partir de volumes massifs de données, ils souffrent néanmoins d'une opacité décisionnelle (boîte noire, propension aux hallucinations, manque de rigueur logique) : en intégrant les principes de l'IA symbolique, fondés sur la manipulation explicite de règles formalisées, d'ontologies et de raisonnements déductifs, l'approche neuro-symbolique agit comme un véritable garde-fou. Elle vise ainsi à doter les modèles connexionnistes d'un cadre d'interprétabilité, de garanties logiques et d'une capacité de contrôle, assurant un comportement à la fois performant, vérifiable et aligné sur des contraintes rationnelles strictes.

# PREMIERE PARTIE : HISTOIRE DE L'IA

## I. L'IA SYMBOLIQUE

L'IA symbolique est la première des branches à se développer à Dartmouth : donner à la machine des concepts clairs et des règles logiques explicites. La penser comme le ferait un raisonnement humain, dans un rendu entièrement formel.



La branche symbolique s'est retirée dans les niches où la rigueur logique reste irremplaçable, avant de revenir, décennie après décennie, corriger les angles morts du connexionnisme.

## II. LES RESEAUX DE NEURONES : DEEP LEARNING, LLMS

Là où le symbolique parie sur la règle explicite, le connexionnisme fait un pari inverse : que l'intelligence puisse émerger d'unités simples, connectées entre elles, ajustant leurs poids au contact de l'expérience, sans qu'aucune règle n'ait jamais à être écrite à la main.



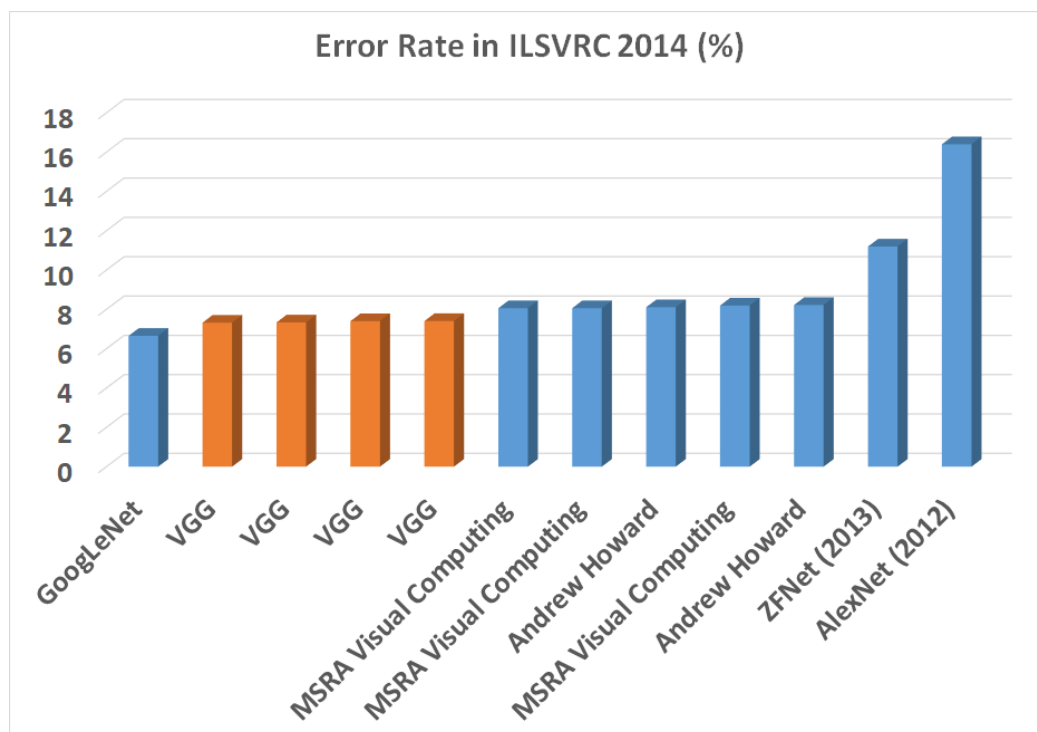
De 1958 à aujourd'hui, le connexionnisme aura mis près de cinquante ans à honorer sa promesse initiale, mais l'a honorée avec un tel éclat qu'il domine aujourd'hui le champ tout entier. Cette réussite a pourtant longtemps cohabité avec une autre voie, plus discrète, qui préférait aux neurones les garanties mathématiques des méthodes statistiques classiques.

## Compétition ImageNet (2012)

L'ImageNet Large Scale Visual Recognition Challenge (ILSVRC), lancée en 2010 : une base de données massive de plus d'un million d'images labellisées dans 1000 catégories. L'objectif de la compétition : une classification d'images avec le meilleur taux d'erreur.

En 2012, Alex Krizhevsky, étudiant de Hinton à Toronto, soumet AlexNet avec Ilya Sutskever et Hinton : Résultat, un écart de presque 11 points sur le second. Une synthèse d'ingrédients, combinés pour la première fois à cette échelle, produisent quelque chose de qualitativement différent :

- Architecture convolutive profonde : 8 couches apprenantes (5 conv + 3 fully connected), directement dans la lignée de LeCun, mais beaucoup plus grande.
- ReLU comme fonction d'activation : à la place de tanh/sigmoïde, ce qui accélère considérablement l'entraînement et atténue le vanishing gradient.
- Dropout : technique de régularisation qui éteint aléatoirement des neurones pendant l'entraînement, forçant le réseau à apprendre des représentations redondantes et robustes, réduisant l'overfitting.
- Data augmentation : transformations aléatoires des images (crops, flips) pour multiplier artificiellement les données d'entraînement.
- GPU : Krizhevsky entraîne le réseau sur deux GTX 580, ce qui rend computationnellement faisable ce qui ne l'était pas sur CPU. C'est peut-être l'élément le plus décisif dans l'immédiat : la puissance de calcul comme facteur de déblocage.

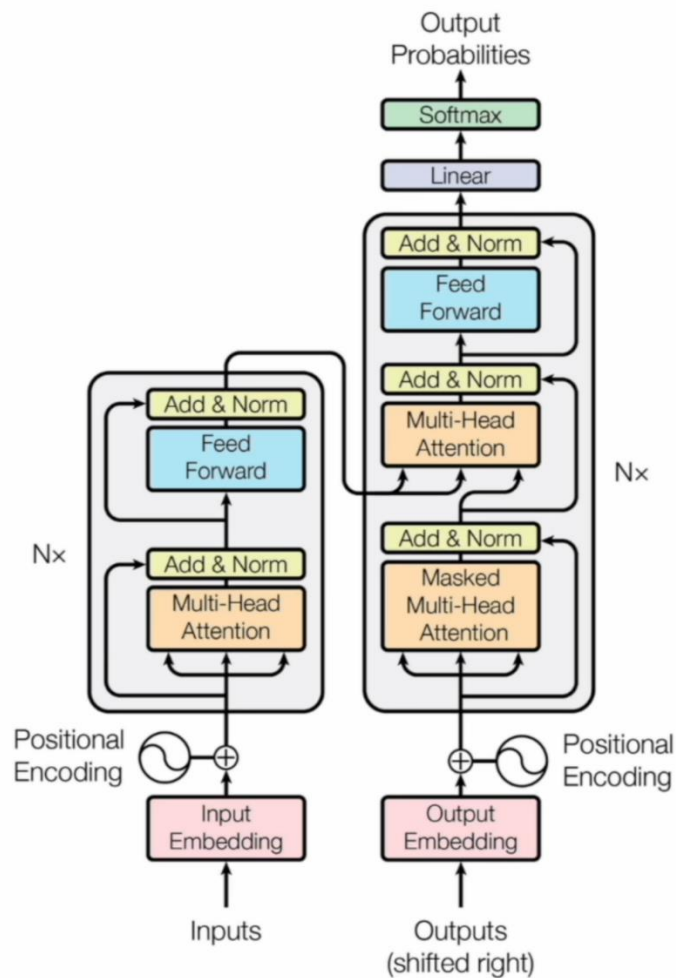


Entre 1986 et 2012, les conditions matérielles ont rattrapé les intuitions théoriques : les données (ImageNet), la puissance de calcul (GPU), et la profondeur des architectures ont convergé au bon moment. La théorie attendait l'infrastructure.

## Attention is all you need (2017)

Publié en 2017 par une équipe de chercheurs de Google, le papier Attention Is All You Need introduit l'architecture Transformer, franchissant une étape décisive dans le traitement du langage naturel en abandonnant les réseaux récurrents au profit des mécanismes d'attention.

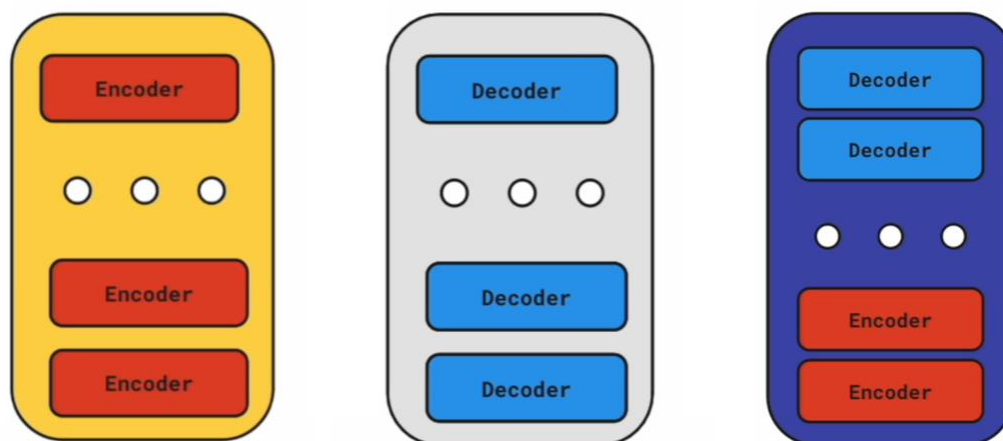
L'architecture transformers est divisée en deux parties distinctes, l'encodeur et le décodeur. Ces composants fonctionnent conjointement et partagent un certain nombre de similitudes.



- **Embeddings** — Les modèles d'apprentissage automatique ne sont que de gros calculateurs statistiques et ils travaillent avec des nombres, pas avec des mots. Avant de transmettre des textes au modèle pour traitement, il faut donc d'abord tokeniser les mots. En termes simples, cela convertit les mots en nombres, chaque nombre représentant une position dans un dictionnaire de tous les mots possibles avec lesquels le modèle peut travailler.
- **Le mécanisme d'attention** — La Self-Attention et le Scaled Dot-Product sont des concepts étroitement liés : l'auto-attention est l'idée générale selon laquelle les tokens portent attention les uns aux autres, tandis que l'attention par produit scalaire mis à l'échelle est une méthode spécifique pour calculer ces poids d'attention avec une mise à l'échelle destinée à assurer la stabilité numérique.

- La **Multi-Head Attention** est une extension du mécanisme d'attention dans les Transformers qui enrichit la Self-Attention et le Scaled Dot-Product.
  - Différentes têtes capturent des motifs et des relations divers dans les données d'entrée : les sorties de toutes les têtes sont concaténées puis transformées linéairement pour produire le résultat final.
  - Cela permet au modèle de se concentrer simultanément sur plusieurs aspects de l'entrée ; il améliore les performances par rapport à l'attention à une seule tête en apprenant des représentations plus riches.
- Le **Position-wise Feed-Forward Network (FFN)** prend la représentation de chaque mot, un par un et séparément, et la fait passer dans un petit réseau de neurones pour enrichir, digérer et approfondir ses informations, sans plus s'occuper des autres mots autour. Le terme Position-wise signifie que le même réseau de neurones est appliqué de la même façon pour chaque position (mot) de la séquence, indépendamment les uns des autres.

L'architecture Transformer originale consiste en une structure encodeur-décodeur. À partir de celle-ci, trois architectures de transformers principales sont apparues (BERT, GPT et T5), qui constituent le fondement de la plupart des modèles basés sur les transformers.



- **Encodeur uniquement (BERT)** — Conçu pour une compréhension bidirectionnelle, il traite les tokens d'entrée simultanément afin de capturer le contexte dans les deux directions. Utilisé pour des tâches comme la classification de texte, le question-réponse et les embeddings (BERT, RoBERTa). Il se concentre sur la modélisation de langage masqué (MLM), où certains tokens sont masqués et le modèle les prédit.
- **Décodeur uniquement (GPT)** — Autorégressif, il traite les tokens séquentiellement pour générer du texte, idéal pour des tâches comme la complétion de texte et le dialogue. Utilisé dans des modèles comme GPT-3, GPT-4 et de nombreux LLM propriétaires (parties de Claude ou LLaMA). Entraîné à la prédiction du token suivant, en se concentrant sur la modélisation de langage causale.
- **Encodeur-décodeur (p. ex. T5)** — Combine un encodeur pour la compréhension de l'entrée et un décodeur pour la génération de la sortie, adapté à des tâches comme la traduction, le résumé et les transformations texte-vers-texte.

### III. L'APPRENTISSAGE AUTOMATIQUE

L'apprentissage automatique parie sur le comportement : peu importe la forme du modèle, seule compte sa capacité à s'améliorer au contact des données, avec des garanties mathématiques que les réseaux de neurones de l'époque ne peuvent pas offrir.



L'apprentissage automatique se fonde presque entièrement à partir de 2012 dans la branche connexionniste, à l'exception des données tabulaires (Gradient Boosting)

## IV. LE TRAITEMENT DU LANGAGE NATUREL (NLP)

Le traitement automatique du langage a successivement intégré la rigueur explicite du courant symbolique, les modèles probabilistes du Machine Learning classique, puis les architectures connexionnistes. L'histoire du langage en IA n'est pas une trajectoire autonome, mais la synthèse évolutive des trois principaux paradigmes.



Des règles de Chomsky aux transformers, le NLP aura connu les mêmes ruptures que les trois branches qui l'ont nourri, sans jamais s'arrêter longtemps sur l'une d'elles.

## DEUXIEME PARTIE : PERSPECTIVES

Le scaling a d'abord paru suffire : plus de paramètres, plus de données, plus de calcul, et les architectures du moment prétendaient absorber, par simple statistique, ce que la vérification externe et les règles symboliques avaient longtemps dû garantir. Cette absorption n'est pourtant jamais totale. Derrière les performances, ressurgissent des exigences de structure, de contrôle et d'action situés, d'où un retour du symbolique, souvent sous forme hybride. La question est de savoir si une fusion neuro-symbolique véritable est possible, ou si le neurone continue d'avaler le symbole sans en conserver la contrainte.

### I. LE PARI DU SCALING : PARAMETRES, DONNEES, CALCUL

Un signal scientifique existe : les grands labos observent des rendements décroissants du pur scaling en paramètres, différentes capacités plafonnant à des tailles de modèle différentes, et les données d'entraînement de haute qualité se raréfient.

Ilya Sutskever, cofondateur et ancien scientifique en chef d'OpenAI, a déclaré lui, lors du NeurIPS de décembre 2024, que "l'ère du scaling telle qu'on la connaît va se terminer" : il évoquait la fin du pré-entraînement tel qu'on le connaît, comparant les données d'internet à un combustible fossile en voie d'épuisement.

- Ce qui a rendu le scaling si influent, c'est qu'elle a été formalisée mathématiquement. L'article fondateur, *Scaling Laws for Neural Language Models*, montre que la perte d'entraînement (l'erreur du modèle) décroît selon une loi de puissance en fonction des trois variables : paramètres, données, calcul.
  - Cette prévisibilité a eu une conséquence stratégique directe : elle a transformé la recherche en IA générative en un problème d'ingénieur plutôt que de découverte scientifique. Il ne s'agissait plus de trouver une architecture radicalement nouvelle, mais de réunir davantage de données, de GPU et de capital — ce qui explique en grande partie pourquoi les grands laboratoires ont pu justifier des investissements aussi massifs : le scaling offrait une feuille de route quasi garantie vers de meilleures performances, à condition d'en payer le prix.
  - Cette loi de puissance a été affinée dès 2022 par les lois de Chinchilla, qui montrent que la plupart des modèles de l'époque étaient en réalité sur-paramétrés par rapport à leurs données, ce qui a lui-même redéfini ce que « bien scaler » signifie. Et c'est cette même loi de puissance qui, selon Sutskever et d'autres, commence à buter non pas sur les paramètres ou le calcul, mais sur la variable la plus difficile à faire croître indéfiniment : la donnée elle-même.

L'argent est déjà engagé sur une hypothèse architecturale (le transformer scale) avant même que la communauté scientifique ait tranché si c'est la bonne voie : ce qui, si des alternatives comme Mamba, JEPA ou une architecture encore inconnue s'avère supérieure dans dix ans, pourrait ralentir l'adoption non pas par manque de mérite scientifique mais par pur verrouillage d'infrastructure (sunk cost à l'échelle de centaines de milliards).

## Les architectures candidates à la relève

Cinq grandes alternatives de recherche à "améliorer le transformer par le scaling"

- L'article fondateur de S4 / Mamba (coécrit avec Albert Gu, Tri Dao) : complexité linéaire au lieu de quadratique, une remise en cause de l'attention elle-même comme mécanisme central.
- World Models / architectures prédictives non génératives (JEPA) : apprendre une représentation du monde par prédiction dans un espace latent plutôt que par génération token par token ; je ne te renvoie pas vers celui qui la porte publiquement, juste vers la direction technique en tant que telle.
- Synthèse de programmes et recherche au moment du test : la critique de François Chollet (créateur d'ARC-AGI) n'est pas juste polémique ; il propose une mesure différente de l'intelligence (généralisation à faible donnée) et des architectures orientées recherche de programme plutôt que mémorisation statistique.
- Apprentissage causal : travaux inspirés de Judea Pearl, repris récemment par Yoshua Bengio (représentations causales), qui cherchent à faire apprendre au modèle une structure causale plutôt qu'une corrélation.
- Alternatives à la rétropropagation elle-même, le Forward-Forward algorithm de Geoffrey Hinton (2022), codage prédictif inspiré des neurosciences (Karl Friston, principe d'énergie libre) : remettre en cause non pas l'architecture mais la règle d'apprentissage.

Les trois premiers (SSM, JEPA, Chollet) remettent en cause l'architecture ou la tâche d'apprentissage, tandis que les deux derniers (causal, Forward-Forward/Friston) remettent en cause quelque chose de plus profond, la règle d'apprentissage elle-même.

## II. UN RETOUR DU SYMBOLIQUE

Le problème concret que le pur connexionnisme peine à résoudre : hallucinations, absence de garanties logiques, raisonnement multi-étapes fragile. Ce que le symbolique peut apporter que l'échelle seule ne peut pas : structure, vérifiabilité, composabilité, - les mêmes qualités que Lighthill, ironiquement, jugeait insuffisantes à elles seules en 1973.

On appelle neuro-symbolique toute architecture qui vient adosser au réseau de neurones une composante logique explicite, qu'elle s'y greffe de l'extérieur, comme un garde-fou, ou qu'elle s'y intègre au point de devenir un rouage du raisonnement lui-même. Ce besoin s'impose à mesure que les limites du pur connexionnisme deviennent visibles : hallucinations, absence de garanties, raisonnement qui s'effondre dès qu'il faut enchaîner plusieurs étapes fiables. Les formes concrètes que prend ce retour aujourd'hui : graphes de connaissances, RAG, agents qui exécutent du code, modèles à recherche arborescente, - autant d'exemples où le symbolique n'affronte pas le connexionnisme mais s'y greffe.

## Hybridation

Cinq tentatives concrètes, aujourd'hui à l'œuvre, permettent de mesurer ce que cette hybridation peut recouvrir, qu'elle reste en périphérie du modèle, ou s'y loge en son cœur.

- Automated Reasoning checks (Amazon Bedrock) et la famille des vérificateurs SMT/PDDL pour les agents (Gideon en robotique), commercialement répandu aujourd'hui : un planificateur ou vérificateur symbolique externe qui garde le LLM en laisse. C'est la forme la plus prudente, la moins intégrée, la plus déployée.
- AlphaGeometry - AlphaGeometry 2 (DeepMind, 2024) : un LM neuronal propose des constructions géométriques auxiliaires (intuition créative), un moteur de déduction symbolique vérifie et complète la preuve. Résultat : niveau médaille d'or aux olympiades de géométrie (IMO). Intégration en boucle, mais les deux modules restent distincts.
- AlphaProof (DeepMind, 2024) : un agent de renforcement type AlphaZero apprend à produire des preuves formelles vérifiées pas à pas par l'assistant de preuve Lean. Ici le symbolique n'est pas un vérificateur a posteriori mais la structure même de l'espace de recherche du RL.
- Neuro-Symbolic Concept Learner (MIT/IBM, 2019) : apprend des concepts visuels (couleur, forme) par perception neuronale, puis exécute un programme symbolique explicite pour répondre aux questions sur une scène. Modèle académique, jamais vraiment sorti du labo.
- Logic Tensor Networks / DeepProbLog: les règles logiques sont rendues différentiables et injectées directement dans la fonction de perte du réseau. C'est le seul des cinq qui vise une vraie fusion architecturale plutôt qu'un pipeline en deux temps.

## Un exemple : l'IA Agentique

Pour l'immense majorité des cas d'usage textuels courants (résumé, rédaction, extraction structurée, support client de premier niveau, code boilerplate), les modèles actuels, ou même plus petits et distillés, sont largement suffisants : pas besoin d'un modèle "frontier" à des centaines de milliards de dollars d'entraînement.

- Une étude du MIT (NANDA, 2025) a montré que 95 % des pilotes de GenAI en entreprise ne produisent aucun impact mesurable, et la cause identifiée n'est quasiment jamais le modèle mais l'intégration (qualité des données, workflows, gouvernance, absence de système où faire atterrir la sortie du modèle). Gartner, de son côté, prévoit que plus de 40 % des projets d'IA agentique spécifiquement seront abandonnés d'ici fin 2027, pour la même raison : ce n'est presque jamais le modèle qui est en cause, c'est l'intégration.
- L'agentique est une innovation d'architecture logicielle autour du modèle (boucles d'appel d'outils, mémoire, planification, découpage de tâches), elle ne nécessite pas un modèle plus puissant : les correctifs qui émergent pour l'agentique ne sont presque jamais "plus de scale", mais des garde-fous symboliques externes (vérification de plan par contrainte, retour arrière sur erreur détectée, appels d'outils vérifiables)

L'agentique a besoin de plus de structure de vérification autour du modèle qu'on ne lui en donne. Ce qu'elle rend indispensable, c'est une forme de vérification déterministe.

### III. VERS UNE VRAIE FUSION NEURO-SYMBOLIQUE ?

Le critère qui distingue une fusion réelle de ce qu'on vient de décrire est précis : il faudrait que la structure logique vive dans la représentation interne elle-même, au moment de l'inférence, sans appel externe, et surtout qu'elle soit systématique : si le modèle "sait" traiter A et B, il doit pouvoir traiter n'importe quelle nouvelle combinaison de A et B jamais vue à l'identique, par composition, pas par ressemblance statistique avec des exemples proches. C'est exactement l'objection qu'un philosophe, Jerry Fodor, opposait au connexionnisme dès 1988 (avec Zenon Pylyshyn) : un réseau de neurones classique peut apprendre "Jean aime Marie" et "Marie aime Paul" séparément, sans que ça garantisse qu'il comprenne la structure "aime" au point de généraliser correctement à n'importe quelle nouvelle paire, alors qu'un système symbolique, par construction, généralise toujours ainsi. Toute tentative de fusion sérieuse doit répondre à cette objection.

Trois pistes concrètes, à des degrés d'ambition croissants :

- Rendre la logique elle-même différentiable, et l'intégrer comme une couche du réseau (pas un module séparé) : c'est ce que font des travaux comme  $\partial$ ILP (differentiable Inductive Logic Programming, Evans & Grefenstette, 2018) ou les Logic Tensor Networks déjà mentionnés. Les opérateurs logiques (ET, OU, implication) sont réécrits comme des fonctions mathématiques dérivables, insérées directement dans le calcul du réseau. Le gradient traverse alors la logique elle-même pendant l'entraînement : la logique n'est plus un juge extérieur, elle fait partie du calcul qui produit la réponse. Mais ça reste, à ce jour, limité à des domaines restreints (raisonnement sur petits graphes, règles simples), jamais passé à l'échelle d'un LLM généraliste.
- Les représentations vectorielles symboliques (Tensor Product Representations, Paul Smolensky, années 90, relancées récemment) : l'idée est de coder une structure symbolique (arbre syntaxique, relation binaire) comme un vecteur unique obtenu par une opération mathématique réversible, de sorte qu'on puisse à la fois faire de l'apprentissage par gradient dessus ET extraire, en principe, la structure symbolique qu'il encode. C'est une tentative de faire cohabiter les deux mondes dans le même objet mathématique, plutôt que dans deux modules qui se parlent.
- Le phénomène de grokking, étudié en interprétabilité mécanistique (Neel Nanda et al., à partir de 2022). On entraîne un petit réseau à faire de l'arithmétique modulaire par pur apprentissage supervisé, sans lui donner aucune règle. Après une longue phase où il semble seulement mémoriser les exemples, il bascule brutalement vers une généralisation parfaite, et quand on inspecte ses poids, on découvre qu'il a spontanément construit, par la seule descente de gradient, un algorithme structuré et systématique (une transformation de Fourier interne) pour résoudre le problème. Personne n'a injecté cette structure : elle a émergé.

L'histoire de l'intelligence artificielle n'est pas une suite de ruptures isolées, mais un mouvement balancier. Après l'ère des règles symboliques, puis l'hégémonie de l'apprentissage statistique et du pur scaling connexionniste, l'IA semble toucher les limites de la seule montée en échelle. L'avenir de la discipline ne réside sans doute ni dans l'abandon du connexionnisme, ni dans un simple retour en arrière, mais dans la capacité à faire émerger, ou à structurer, une véritable compositionnalité logique au cœur des modèles.

## ANNEXE : DE LA VERIFICATION A L'ABSORPTION

Un phénomène récurrent dans l'histoire de l'IA est l'absorption progressive des contraintes symboliques par l'entraînement neuronal. La vérification ne reste pas nécessairement un module externe distinct : elle tend à être absorbée dans les poids du modèle lui-même par apprentissage par renforcement (RL) sur des tâches vérifiées. Le symbolique disparaît ainsi comme composant visible, sans que le problème de rigueur qu'il résolvait n'ait disparu pour autant. Ce processus se déroule en trois étapes.

**Étape 1** — Le point de départ : la vérification comme filtre externe.

Avant tout apprentissage, il faut d'abord un vérificateur qui existe indépendamment du modèle et qui rend un verdict binaire, fiable, automatique, sans ambiguïté possible. Pour les maths, c'est **Lean** : on lui soumet une preuve écrite dans son langage formel, et il répond "valide" ou "invalid", sans jugement de goût, sans zone grise. C'est un juge extérieur au réseau de neurones : le modèle génère, un système symbolique séparé tranche.

À ce stade, avant toute idée de RL, le vérificateur ne change rien à l'intérieur du modèle. Il ne fait que dire, après coup, si la sortie était bonne ou mauvaise.

**Étape 2** — Distinguer deux formes de "juge", puis comprendre comment le verdict devient un apprentissage.

LLM as a judge et Lean ne sont pas la même chose. Le LLM as a judge utilise un autre modèle de langage pour noter la sortie : c'est un jugement approximatif, une opinion statistique sur ce qui semble bon. Lean ne juge pas, il vérifie : le résultat est mathématiquement certain, binaire, sans marge d'interprétation. C'est cette différence qui garde le côté symbolique dans l'histoire : le vérificateur n'apprend rien et ne se trompe jamais dans son domaine, contrairement à un LLM-juge qui peut se tromper comme n'importe quel modèle.

Maintenant, comment ce verdict devient un apprentissage : le modèle, avant tout entraînement RL, est déjà une machine qui produit des séquences de tokens avec des probabilités (c'est ce qu'on appelle une "politique" en RL). L'idée du RL ici est simple dans son principe, même si les détails mathématiques sont plus subtils :

1. Le modèle génère une tentative complète de preuve.
2. Lean répond : valide (récompense = 1) ou invalide (récompense = 0).
3. L'entraînement ajuste les poids du modèle pour augmenter la probabilité qu'il génère des séquences de tokens similaires à celles qui ont obtenu récompense = 1, et diminuer la probabilité de celles = 0. Le modèle n'apprend pas la règle de Lean, on le renforce statistiquement.
4. Des tactiques de preuve : par exemple, privilégier de découper un problème complexe en sous-cas (raisonnement par disjonction), ou reconnaître qu'une certaine forme d'énoncé se prête à une preuve par récurrence plutôt qu'une preuve directe.

5. Des habitudes de vérification intermédiaire : le modèle peut apprendre à générer une étape, la "recontrôler" mentalement dans sa propre séquence de tokens avant de continuer (poser une sous-conclusion et la reformuler), parce que statistiquement, les tentatives qui font ça arrivent plus souvent à une preuve validée que celles qui foncent tout droit.
6. Des réflexes de correction : reconnaître, dans sa propre génération, un pattern qui ressemble à une impasse déjà rencontrée (même partiellement) dans les données d'entraînement, et changer de direction avant même d'aller au bout : comme un joueur d'échecs qui abandonne une ligne de jeu à mi-chemin parce qu'elle "sent" mauvais, sans avoir calculé jusqu'au mat.
7. Un vocabulaire de lemmes et de constructions utiles dans certains contextes : savoir que tel type de problème invoque typiquement tel outil.

Donc ce n'est pas UN raisonnement privilégié, c'est une distribution de probabilités sur des milliers de micro-stratégies, chacune renforcée ou affaiblie selon sa corrélation historique avec un verdict "valide" de Lean sur des millions d'exemples. Le modèle ne retient pas une règle ("fais X"), il retient une tendance statistique diffuse à produire des séquences qui ressemblent, structurellement, à celles qui ont fonctionné.

**Étape 3** — Ce qui se passe à l'inférence, quand Lean n'est plus là.

Une fois l'entraînement terminé, on pose une nouvelle question au modèle en production : Lean n'est généralement plus présent du tout au moment où le modèle répond. Le modèle génère sa réponse token par token, en utilisant uniquement les poids qu'on vient de décrire : cette distribution de probabilités façonnée par des millions de cycles de vérification passés.

Deux conséquences importantes découlent de ça : le résultat n'est plus garanti (probabilité empirique, sur la base de sa distribution apprise) ; en pratique, on ne supprime pas complètement Lean à l'inférence.

Résultat net : on a deux couches très différentes, un modèle dont les poids ont été façonnés par la logique sans la contenir, et un vérificateur externe qui sanctionne le résultat sans participer à sa production.